Logical English 2: A Reimplementation

Miguel Calejo · Jacinto A. Dávila Quintero

ainexus.com, logicalcontracts.com, Universidad de los Andes

PEG 2026 · ICLP / FLoC · Lisbon · 19 July 2026

Logical English — continued from this morning

- **The text is the program — no separate codification to keep faithful to the prose**
- Templates define the vocabulary; indentation gives and/or scope; semantics: logic programming
- Every answer — and every failure — is explained in the same English

```
the templates are:
   *a person* acquires British citizenship on *a date*.
   *a person* is born in *a place* on *a date*.  ...

the knowledge base citizenship includes:

a person acquires British citizenship on a date
if the person is born in the UK on the date
   and the date is after commencement
   and an other person is the mother of the person
      or the other person is the father of the person
   and the other person is a British citizen on the date
      or the other person is settled in the UK on the date.
```


Logical Contracts

LE1 → LE2: consolidation, not extension

LE1 — research breadth (Imperial College & partners)

- SWISH package · 3 back-ends: Prolog, TaxLog, s(CASP) · English, French, Italian, Spanish
- Events & fluents, collections, deontics… — nearly every idea prototyped at least once

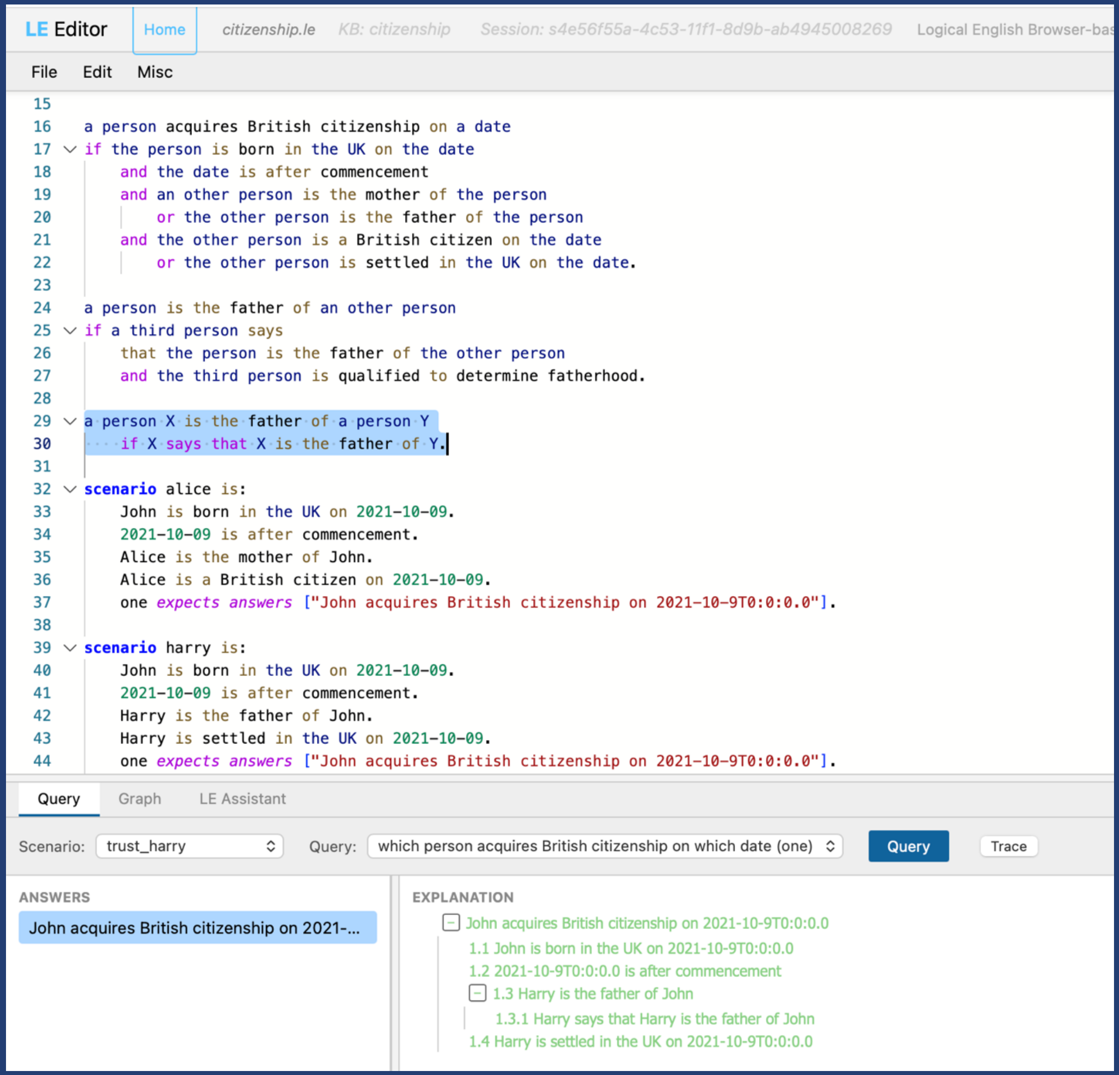
LE2 — a self-contained system (same working group)

- One command or one Docker image: reasoner + browser IDE + web/MCP/DAP APIs
- Single Prolog back-end: meta-interpreter, explanations built in
- Tests and verification are part of the language
- Extension hooks (parse / second pass / reasoning)

Two phases of one project: LE1 explored the design space; LE2 ships its stable core.

A self-contained IDE

- Template-aware editing: your vocabulary drives highlighting, completion, hover
- A verifier at every load: structured diagnostics with suggested fixes
- **Every answer explained — and every failure too**
- Click a proof step → the rule that produced it
- Isolated sessions per document: a classroom on one server
- Graph view · in-editor debugger · MCP for tool-using LLMs



Logical Contracts

Demo 1 · The British Nationality Act, live

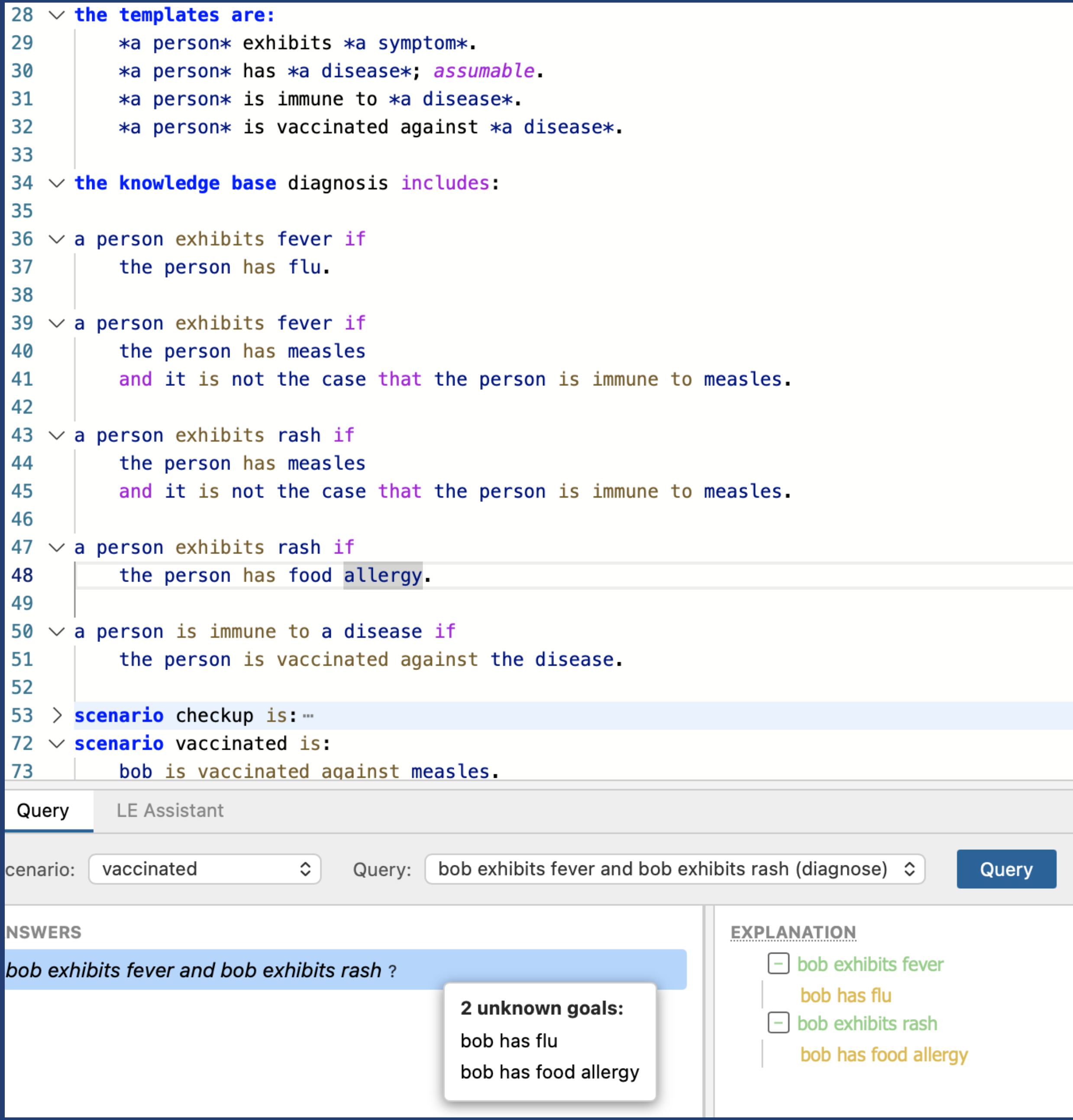
- Read the law — it is the program
- Ask a question; walk the explanation back to the source
- Break the scenario, then repair it from the failure explanation itself
- (what-if by right-click: delete a fact · add it back · assume it)

Abduction

```
the templates are:
   *a person* exhibits *a symptom*.
   *a person* has *a disease*; assumable.

a person exhibits fever if
   the person has measles
   and it is not the case that
      the person is immune to measles.
```

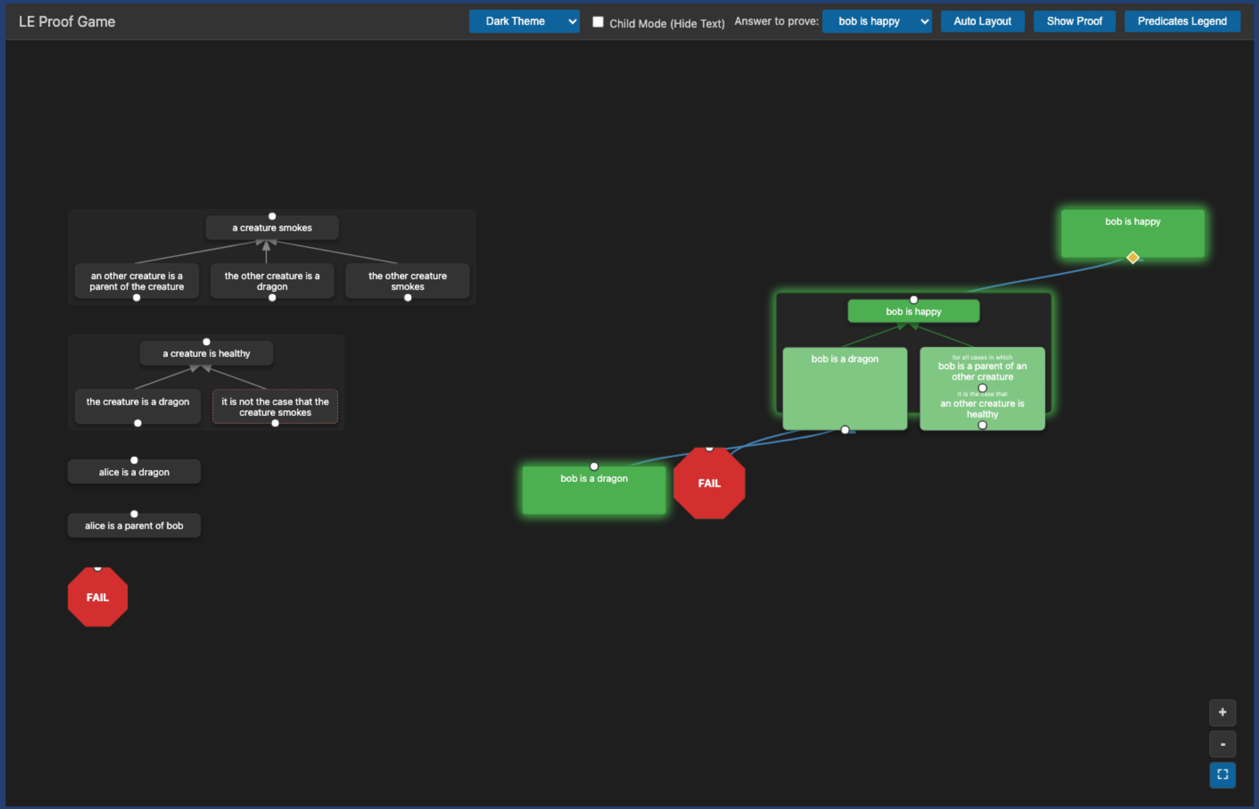
- “; assumable” is the entire syntax — unproven goals are assumed, reported as unknowns
- Conditional answers, as this morning:
- “bob exhibits fever, assuming bob has measles”
- Everywhere: amber explanation nodes, Assume checkboxes, assumption cards
- **No integrity constraints: NAF guards over closed facts eliminate candidates**



Logical Contracts

The Proof Game – for any program

- **This morning's game was propositional — the full game is first-order**
- Wiring binds variables; bind one two ways → a visible clash
- Negation as failure: drop a FAIL card — or wire out why every attempt fails
- “for all cases” must be discharged case by case
- Child (unique colors) Mode; propositional



Demo 2 · Playing a proof

- Happy dragons: wire the proof, meet a clash, FAIL a vacuous “for all”
- Child Mode, for ages before text
- Differential diagnosis: rival explanations — until vaccination eliminates one

Logical Contracts

Since the paper — authoring & LLMs

Authoring without syntax

- Fill-in-the-blank Scenario & Query form editors
- What-if Scenario Variations; patch the scenario
- "Executive mode": read-only, phone-friendly, shareable URL
- Bento boxes

LLM proposes, logic curates

- “Write it in English”: NL → facts/queries through your templates only
- Every proposal verified against the program before insertion; every LE file carries its own tests
- LE Assistant in the IDE

· MCP so agents load, query and verify LE through a typed interface

Using it

- **Public server: le2.logicalcontracts.com**
- Yours: docker run -p 3050:3050 logicalcontracts/le2 — or one SWI-Prolog command
- github.com/LogicalContracts/LogicalEnglish2
- Tutorial · Proof Game teacher's guide · examples library (incl. R. Kowalski's book subset)
- Paper appendix (with later updates)
- Tell us about your use: mc@logicalcontracts.com, jd@logicalcontracts.com

Obrigado — questions?

…just one more thing…

Logical Contracts